

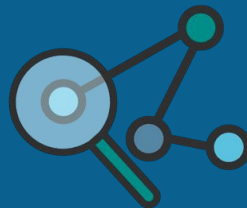


Hands-On With the Neo4j Graph Platform Session 7

Class Agenda

- Labeled Property Graph (LPG) Components 🙌
 - What is a graph database? 🙌
 - What is Neo4j? 🙌
 - Common Graph Use Cases 🙌
 - Graph modeling -Part 1 🙌
 - Graph modeling -Part 2 🙌
- **Importing CSV into Neo4j**
- Constraints and Indexes
 - Intro to Graph Data Science
 - Graph Data Science workflows

🙌 Hands-On





Importing CSV into Neo4j



Neo4j Importing / Exporting

MANY options exist to import data into Neo4j.

Supported

- Cypher LOAD CSV
- Bulk CLI Import (neo4j-admin import)
- Neo4j Data Importer
- APOC Library
 - Web-APIs (file, http, s3, etc)
 - CSV, JSON, XLS, HTML
- Connectors
 - Apache Spark
 - Apache Kafka
- BigQuery
- Google Cloud / Dataflow

Community

- APOC Extended
 - Web-APIs (file, http, s3, etc)
 - JSON, CSV, XML, GraphML
- Database Integrations
 - JDBC, Couchbase, MongoDB, etc
- NeoConverse (RDF)

ETL Tools

- Apache Hop
- Talend
- Informatica
- etc.

Drivers

- Python
- Javascript
- Java
- etc.

Data Importer: Getting started with modeling

Frictionless Experience



Easiest way to model your first graph and load it using flat files

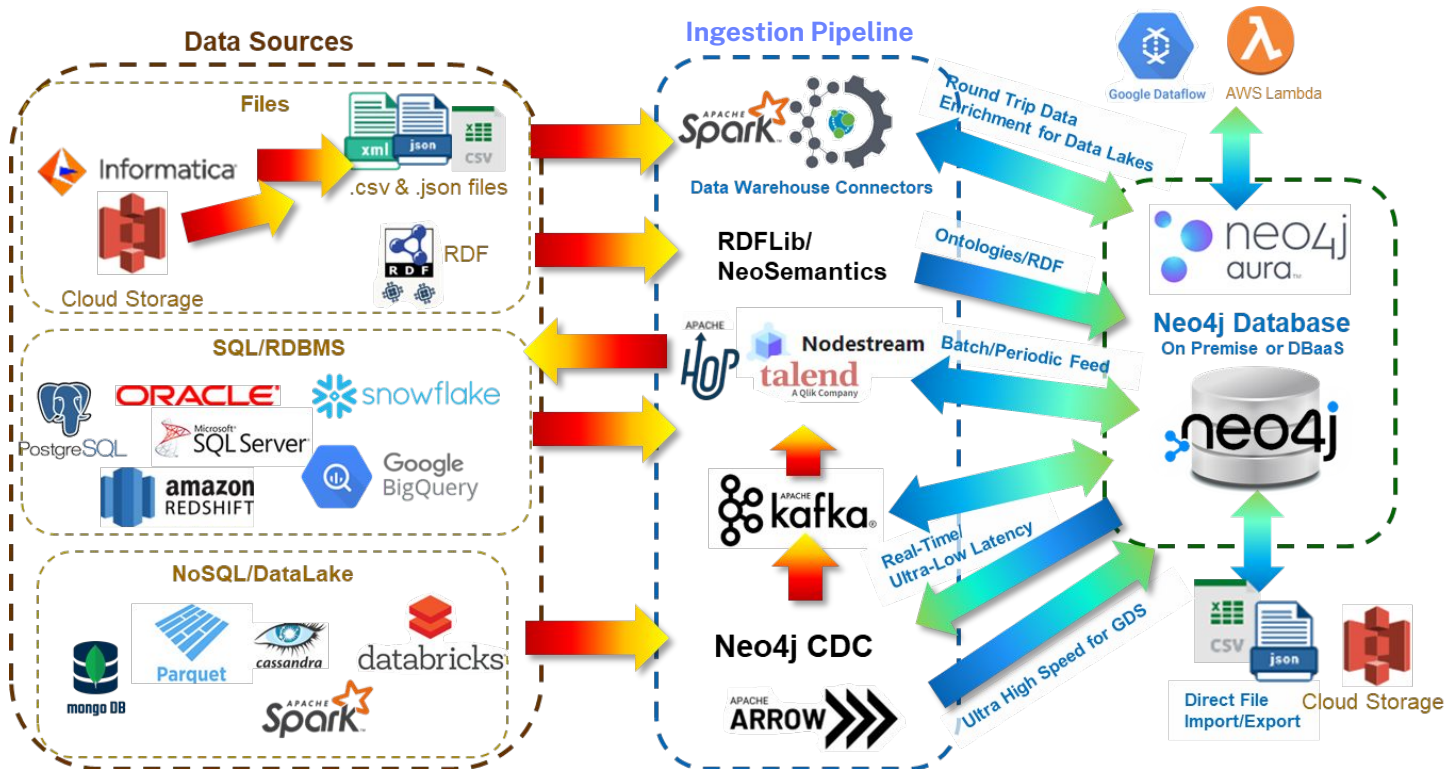
The screenshot displays the Neo4j Data Importer web application. On the left, a 'Files' panel lists several CSV files: 'use_case.csv', 'expertise.csv', and 'attendees.csv'. The central area shows a graph with two nodes, 'Attendee' and 'Use', connected by a dashed line. On the right, the 'Mapping Details' panel is visible, showing fields for 'Label' (set to 'Use'), 'File' (a dropdown menu), 'Properties' (with 'Definition' and 'Mapping' tabs), and 'ID' (a dropdown menu). Buttons for 'Add node', 'Show Results', and 'Run Import' are also present.

- **Intuitive** data modeling UI
- **Map** flat files to graphs
- Built to import up to **millions of records**
- **Import directly** Neo4j Aura or self-managed

<https://medium.com/neo4j/importing-your-data-into-neo4j-just-got-even-easier-3712e531ca70>

<https://neo4j.com/docs/data-importer/current/overview/>

The possibilities in a picture



Commonly Used Solutions

- **LOAD CSV**

- Cypher command
- 10K-50K/sec load speeds
- Good for small/test loads
- Data does not have to be in graph schema

- **Neo4j-admin import**

- Offline bulk load
- 10x faster than LOAD CSV
- Full (initial) or incremental
- Data must be in graph schema
 - CSV files that match node/relationships
 - Best used with ETL tool to get files in right format/schema

- **SparkConnector**

- Used with SPARK sources such as Databricks as low code implementation
- Load speed 20K-50K/sec as it depends on cypher ala Load CSV
- Batching and parallelization built in



Loading CSV into Neo4j



Data Import Process

Before importing, there is **no graph data structure**. Only an **empty database!!!**

Read the
source data

Transform
the data

Create the
nodes,
relationships,
and properties

Normalized data

Each file represents a table
Joins across files using unique IDs

De-normalized data

Single CSV file containing all the data
Data duplicated where there are relationships

Loading CSV Syntax

Syntax

```
LOAD CSV [WITH HEADERS]  
FROM <url> [AS alias]  
[FIELDTERMINATOR char]
```

Special functions

- Linenumber()
- Filename()

Example

```
LOAD CSV WITH HEADERS  
FROM 'https://data.neo4j.com/importing-cypher/people.csv' AS row  
RETURN row limit 10
```

<https://neo4j.com/docs/getting-started/data-import/csv-import/>

Creating nodes, relationships, and properties

Example

```
LOAD CSV WITH HEADERS
FROM 'https://data.neo4j.com/importing-cypher/persons.csv' AS row
MERGE (p:Person {tmdbId: toInteger(row.person_tmdbId)})
SET
  p.imdbId = toInteger(row.person_imdbId),
  p.bornIn = row.bornIn,
  p.name = row.name,
  p.bio = row.bio,
  p.poster = row.poster,
  p.url = row.url,
  p.born = row.born,
  p.died = row.died
```

Case Sensitivity

Labels, Relationship Type
names and Property
names are case-sensitive

Best Practice!!!

LabelNames
CONNECTIONS
propertyNames

<https://neo4j.com/docs/getting-started/data-import/csv-import/>

Using MERGE not CREATE

Syntax

```
MERGE (var:Label {prop1: $value1})  
ON CREATE SET var.prop2=$value2  
ON MATCH SET var.prop3=$value3
```

```
MATCH (a:Label {n: $value1}),  
      (b:Label {n: $value2}),  
MERGE (a)-[:KNOWS]->(b)
```

MERGE allows you to run the Cypher statement multiple times without creating duplicate nodes.

MERGE first performs a **MATCH**.

If found, Cypher will fill the variable with the **MATCH**

If not found, Cypher will create the node or relationship

<https://neo4j.com/docs/getting-started/data-import/csv-import/>

Unique IDs and Constraints



Syntax

```
CREATE CONSTRAINT [name] [IF NOT EXISTS]
FOR (x:LabelName)
  REQUIRE x.propertyName IS [NODE] UNIQUE
[OPTIONS "{" option: value[, ...] "}"]
```

Benefits

- **Duplicate** data is **not created**
- **Prevent problems** creating relationships between nodes
- Creating a constraint **automatically** also creates an **index**

Data Types

LOAD CSV returns everything as a string!!!

Cast data to appropriate data type either *during load or after* (aka. ELT)

Data Types

- String
- Long (integer values)
- Float (decimal values)
- Boolean
- Date / Datetime
- Point (spatial)
- Lists

Casting Functions

- toString()
- toInteger()
- toFloat()
- toBoolean()
- date()
- datetime()

<https://neo4j.com/docs/cypher-manual/current/values-and-types/casting-data/>

Loading Lists

All values in a list must have the same data type.

- ["Apple", "Banana", "Orange"]
- [100, 50, 38]

Split data into a list using the `split()` function.

```
LOAD CSV WITH HEADERS
FROM 'https://data.neo4j.com/importing-cypher/movies.csv' AS row
MERGE (m:Movie {movieId: toInteger(row.movieId)})
SET m.countries = split(row.countries, '|')
```

Inspecting Data Types

Use `db.schema.nodeTypeProperties()` function to review node property data types.

```
CALL db.schema.nodeTypeProperties()
```

```
YIELD nodeType, propertyName, propertyTypes
```

```
neo4j$ CALL apoc.meta.nodeTypeProperties() YIELD nodeType, propertyName, propertyTypes
```

	nodeType	propertyName	propertyTypes
3	":Movie"	"released"	["Long"]
4	":Person"	"name"	["String"]

NOTE: `db.schema.relTypeProperties()` for relationship properties

<https://neo4j.com/docs/operations-manual/current/reference/procedures/>

Labels

- Specify Label with **CREATE** or **MERGE**
- Refactoring the graph and adding Labels using **SET**

Ex1:

```
MATCH (p:Person)-[:ACTED_IN]->()  
WITH DISTINCT p SET p:Actor
```

Ex2:

```
MATCH (p:Account) WHERE p.status="Open"  
SET p:OpenAccount
```



A Walk Thru Example



Examples of Inspecting the Data

View a couple of rows to find header names

```
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input  
RETURN count(input)
```

How large
is the data?

View a couple of rows to find header names

```
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input  
RETURN linenumber(), input  
SKIP 10000 LIMIT 500
```

Check the
format of
the data

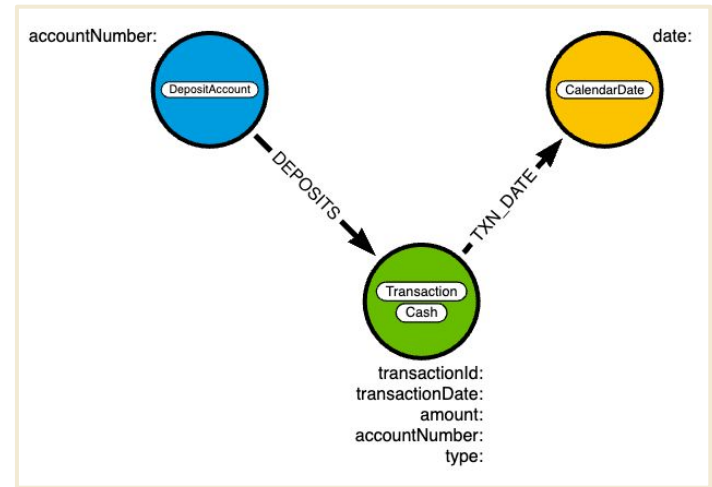
LOAD CSV Example - Typical

// Step 1 - Load the data - if we know this is “new” data, we can just call CREATE vs. MERGE

```
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input
MATCH (myDate:CalendarDate {date: date(input.transaction_date)})
MATCH (myAcct:DepositAccount {accountNumber: input.account_number})
CREATE (txn:Cash:Transaction {
    transactionId:      input.transaction_id,
    transactionDate:    input.transaction_date,
    amount:             toFloat(input.amount),
    accountNumber:      input.account_number,
    type:               "DEPOSIT"
})
MERGE (txn)-[:TXN_DATE]->(myDate)
MERGE (myAcct)-[:DEPOSITS]->(txn)
```



**All data loaded within
1 transaction!!!!**



Using Multiple Transactions



Best Practice!!!

Split a query into multiple transactions:

Introduced
in v5.23

```
MATCH (p:Account) WHERE p.status="DEPOSIT"  
CALL (p) {  
    SET p:OpenAccount  
} IN TRANSACTIONS OF 1000 ROWS
```

Introduced
in v4.x

```
MATCH (p:Account) WHERE p.status="DEPOSIT"  
CALL {  
    WITH p  
    SET p:OpenAccount  
} IN TRANSACTIONS OF 1000 ROWS
```

Removed
in v5.x

USING PERIODIC COMMIT

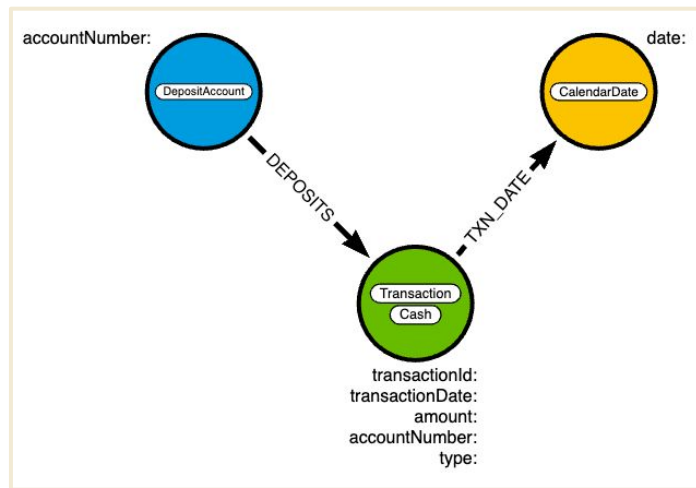
https://neo4j.com/docs/cypher-cheat-sheet/5/aura-dbe/#_call_subqueries_in_transaction

LOAD CSV Example - With Transactions (Better)

```
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input
MATCH (myDate:CalendarDate {date: date(input.transaction_date)})
MATCH (myAcct:DepositAccount {accountNumber: input.account_number})
CALL (myDate, myAcct) {
  CREATE (txn:Cash:Transaction {
    transactionId:      input.transaction_id,
    transactionDate:    input.transaction_date,
    amount:             toFloat(input.amount),
    accountNumber:      input.account_number,
    type:               "DEPOSIT"
  })
  MERGE (txn)-[:TXN_DATE]->(myDate)
  MERGE (myAcct)-[:DEPOSITS]->(txn)
} IN TRANSACTIONS OF 1000 ROWS
```



Writing and then reading data within the same transaction may result in an Eager operator or deadlocks



Running multiple queries

Separate statements using semicolon

```
MATCH (p:Person)-[:ACTED_IN]->()  
WITH DISTINCT p SET p:Actor;
```

```
MATCH (p:Account) WHERE p.status="DEPOSIT"  
SET p:OpenAccount;
```

LOAD CSV Example - Multi-Step (Most Optimal?)

Step 1

```
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input
MATCH (myDate:CalendarDate {date: date(input.transaction_date)})
MATCH (myAcct:DepositAccount {accountNumber: input.account_number})
CALL (myDate, myAcct) {
    CREATE (txn:Cash:Transaction {
        transactionId:      input.transaction_id,
        transactionDate:    input.transaction_date,
        amount:             toFloat(input.amount),
        accountNumber:      input.account_number,
        type:                "DEPOSIT"
    })
} IN TRANSACTIONS OF 1000 ROWS;
```

:auto required if running from the Neo4j Browser

Step 2

```
LOAD CSV WITH HEADERS FROM 'file:///atm_txns.csv' AS input
MATCH (myDate:CalendarDate {date: date(input.transaction_date)})
MATCH (myAcct:DepositAccount {accountNumber: input.account_number})
MATCH (txn:Transaction {transactionId: input.transaction_date})
CALL (myDate, myAcct, txn) {
    MERGE (txn)-[:TXN_DATE]->(myDate)
    MERGE (myAcct)-[:DEPOSITS]->(txn)
} IN TRANSACTIONS OF 1000 ROWS;
```


Cleaning up the graph

Separate statements using semicolon

Just relationships ⇒ `MATCH (p:Person)-[r:ACTED_IN]->() DELETE r;`

Just nodes ⇒ `MATCH (p:Person) DELETE p;`

Both ⇒ `MATCH (p:Person) DETACH DELETE p;`

*Error if nodes
have
relationships*

Example for deleting large amounts of data (Ex: relationships)

```
MATCH (p:Person)-[r:ACTED_IN]->()
CALL (r) {
  DELETE r
} IN TRANSACTIONS OF 1000 ROWS
```



Real World Example



Real world (POC load of small data set) (1)

// Node Keys

```
CREATE CONSTRAINT Brand_nkey IF NOT EXISTS FOR (n:Brand) REQUIRE (n.productName) IS NODE KEY;  
CREATE CONSTRAINT HCP_MDM_ID_nkey IF NOT EXISTS FOR (n:HCP) REQUIRE (n.MDM_ID) IS NODE KEY;  
CREATE CONSTRAINT Geo_nkey IF NOT EXISTS FOR (n:Geo) REQUIRE (n.geoValue, n.geoType) IS NODE KEY;  
CREATE CONSTRAINT GeoProfile_nkey IF NOT EXISTS FOR (n:GeoProfile)  
    REQUIRE (n.geoName, n.MDM_ID, n.productName, n.IC, n.useCase) IS NODE KEY;  
CREATE CONSTRAINT Specialty_nkey IF NOT EXISTS FOR (n:Specialty) REQUIRE (n.specialty) IS NODE KEY;  
CREATE CONSTRAINT Tier_nkey IF NOT EXISTS FOR (n:Tier) REQUIRE (n.tier) IS NODE KEY;  
CREATE CONSTRAINT RxHistory_nkey IF NOT EXISTS FOR (n:RxHistory)  
    REQUIRE (n.brand, n.productName, n.MDM_ID, n.weekEnding, n.submetric) IS NODE KEY;
```

// Indexes

```
CREATE INDEX HCP_fullName_idx IF NOT EXISTS FOR (n:HCP) ON (n.fullName);  
CREATE INDEX HCP_NPI_idx IF NOT EXISTS FOR (n:HCP) ON (n.NPI);
```

Real world (POC load of small data set) (2)

```
:auto
LOAD CSV WITH HEADERS FROM 'file:///<somefilename>.csv' as inputRow
WITH inputRow
CALL {
  WITH inputRow // pre-Neo4j 5.23.0 style
  MERGE (g:Geo {geoValue: inputRow.GEO_VALUE, geoType: inputRow.GEO_TYPE})
  MERGE (s:Specialty {specialty: inputRow.SPEC})
  MERGE (t:Tier {tier: inputRow.TIER})
  MERGE (hcp:HCP {MDM_ID: inputRow.HCP_MDM_ID})
  ON CREATE SET hcp.NPI=inputRow.HCP_NPI
    , hcp.fullName=inputRow.HCP_NAME
    , hcp.specialty=inputRow.SPEC
    , hcp.tier=inputRow.TIER
  MERGE (prof:GeoProfile {IC: inputRow.IC, MDM_ID: inputRow.HCP_MDM_ID, geoName: inputRow.GEO_VALUE
    , productName: inputRow.PRODUCT, useCase: inputRow.USE_CASE_NAME})
  ON CREATE SET prof.target=(CASE WHEN toUpper(inputRow.TARGET)="Y" THEN true ELSE false END)
    , prof.firstDegreeInfluencerScore = toFloat(inputRow.FIRST_DEGREE_INFLUENCER_SCORE)
    , prof.pageRankInfluencerScore = toFloat(inputRow.PAGE_RANK_INFLUENCER_SCORE)
    , prof.geoScore = toFloat(inputRow.GEO_SCORE)
    , prof.geoRank = toInteger(inputRow.GEO_RANK)
    , prof.nationalScore = toFloat(inputRow.NATIONAL_SCORE)
    ...
    , prof.affiliationConnections = toInteger(inputRow.AFFILIATION_CONNECTIONS)
    , prof.referralConnections = toInteger(inputRow.REFERRAL_CONNECTIONS)
  MERGE (hcp)-[:HAS_PROFILE]->(prof)
} IN TRANSACTIONS
;
```

Real world (POC load of small data set) (3)

```
:auto
LOAD CSV WITH HEADERS FROM 'file:///<somelocalfilename>.csv' as inputRow
WITH inputRow
    , toInteger(substring(inputRow.YRWK,0,4)) as year
    , toInteger(substring(inputRow.YRWK,4,2)) as weekNum
WITH inputRow, date({year: year, week: weekNum})+duration({days: 4}) as myDate
MATCH (hcp:HCP {MDM_ID: inputRow.HCP_MDM_ID})
CALL {
    WITH inputRow, hcp, myDate
    MERGE (rx:RxHistory {MDM_ID: inputRow.HCP_MDM_ID, brand: inputRow.BRAND
        , productName: inputRow.PRODUCT_NAME, weekEnding: myDate
        , submetric: inputRow.SUBMETRIC})
    ON CREATE SET rx.TRX=toFloat(inputRow.TRX)
        , rx.NRX=toFloat(inputRow.NRX)
    MERGE (hcp)-( :HAS_RX_HISTORY)->(rx)
} IN TRANSACTIONS OF 1000 ROWS
;
```

Real world (POC load of small data set) (4)

```
:auto
// chain the sequence together
MATCH (hcp:HCP)-[:HAS_RX_HISTORY]->(rx:RxHistory)
WITH hcp, rx
ORDER BY rx.weekEnding
WITH hcp, collect(rx) as historyList
WITH hcp, historyList, size(historyList) as cnt
CALL {
    WITH hcp, historyList, cnt
    WITH hcp, historyList, historyList[0] as firstRX, historyList[-1] as lastRX, cnt
    MERGE (hcp)-[:FIRST_RX_HISTORY]->(firstRX)
    MERGE (hcp)-[:LAST_RX_HISTORY]->(lastRX)
    WITH hcp, historyList, range(1,cnt-1) as listIdx
    WHERE cnt>1
    UNWIND listIdx as curIdx
    WITH hcp, historyList[curIdx-1] as prevRx, historyList[curIdx] as curRx
    MERGE (prevRx)-[:NEXT_RX_HISTORY]->(curRx)
}IN TRANSACTIONS OF 1000 ROWS
;
```



Importing data considerations



What if the CSV has no headers?

// if headers are not present, then map contents can be addressed like an array

```
LOAD CSV FROM 'file:///atm_txns.csv' AS inputRow
```

// first column in file is index=0

```
WITH inputRow[0] as aliasName1
```

// cast data to desired datatype and give a meaningful alias name

```
, toInteger(inputRow[1]) as aliasName2
```

```
, date(inputRow[2]) as aliasName3
```

...

```
CREATE (n:Node {
```

```
  columnName1:    aliasName1
```

```
, columnName2:  aliasName2,
```

```
, columnName3:  aliasName3,
```

...

```
})
```


What if the headers have spaces?

```
LOAD CSV WITH HEADERS FROM 'file:///Retail_Trade_Part_3.csv' AS input
CREATE (m:Merchant {
    merchantID:                toInteger(input.CompanyID),
    merchantName:               input.`COMPANY NAME`,
    webAddress:                 input.`WEB ADDRESS`,
    creditRating:               input.`CREDIT ALPHA SCORE`,
    contactLastName:            input.`LAST NAME`,
    contactFirstName:           input.`FIRST NAME`
});
```

Use backtick
for quotes

Could also
use aliases

What if a relationship is optional?

Filter out
rows where
there is no
relationship

```
LOAD CSV WITH HEADERS FROM 'file:///Retail_Trade_Part_3.csv' AS input  
WITH input  
WHERE NOT input.`PRIMARY SIC` IS NULL
```

```
MATCH (m:Merchant {merchantID: toInteger(input.CompanyID)})  
MERGE (sic:BusinessClassification {SIC_CODE: toInteger(input.`PRIMARY SIC`)})  
    ON CREATE SET sic.description=input.`PRIMARY SIC DESCRIPTION`  
MERGE (m)-[:HAS_SIC]->(sic);
```

What slows down loading?

A lot of MERGE statements

- If single label with a lot of relationships
 - ✓ Maybe process in groups of relationships
- If a lot of labels being created
 - ✓ Split into separate loads per label

A lot of properties

- If more than 5 – especially more than 10 properties per label

Running out of heap memory

- E.g. deleting or creating millions of nodes in **single transaction**
 - ✓ Use `CALL { ... } IN TRANSACTIONS OF X ROWS`

Best Practices Summary



- **Naming conventions improve usability for non-technical users**
 - LabelNames (Pascal Case)
 - RELATIONSHIPS (All capitalized)
 - propertyNames (Camel Case)
- **Create constraints prior to loading data**
 - Improves performance on MATCH & MERGE
- **Use multiple transactions CALL {...} IN TRANSACTIONS OF X ROWS**
 - Reduces memory use and potential deadlocks
- **Load data using multiple passes through the file(s)**
 - Firstly create nodes
 - Then load relationships



Admin import



Neo4j-admin import example script (Windows)

```
set NEO4J_HOME=C:\neo4j\neo4j-enterprise-5.23.0
set HEAP_SIZE=12G
set NODES_DATA=H:\test_data\load_test\bbn\nodes
set RELS_DATA=H:\test_data\load_test\bbn\relationships
```

```
%NEO4J_HOME%\bin\neo4j-admin.bat database import incremental --force ^
--nodes=Customer=%NODES_DATA%\Customer\part-00000-tid-7355961353544110083.csv ^
--nodes=Vendor=%NODES_DATA%\Vendor\part-00000-tid-2503382915754629370.csv ^
--nodes=SalesOrder=%NODES_DATA%\SalesOrder\part-00000-tid-5677790003783795461.csv ^
--nodes=PurchaseOrder=%NODES_DATA%\PurchaseOrder\part-00000-tid-7846597396141780798.csv ^
--nodes=MfgOrder=%NODES_DATA%\MfgOrder\part-00000-tid-3582336956172245584.csv ^
--nodes=MatlDoc=%NODES_DATA%\MatlDoc\part-00000-tid-89012482034361627.csv ^
--nodes=MatlBatchPlant=%NODES_DATA%\MatlBatchPlant\part-00000-tid-7794671700393235023.csv ^
--relationships=BATCH_MFG=%RELS_DATA%\BATCH_MFG\part-00000-tid-3721076039084795102.csv ^
--relationships=BATCH_SOLD_TO=%RELS_DATA%\BATCH_SOLD_TO\part-00000-tid-4697518520788859970.csv ^
--relationships=BATCH_TRANSFER=%RELS_DATA%\BATCH_TRANSFER\part-00000-tid-5329238344437313631.csv ^
--relationships=CONSUMED_IN_ORDER=%RELS_DATA%\CONSUMED_IN_ORDER\part-00000-tid-5185759976999552206.csv ^
--relationships=PO_CONTAINS=%RELS_DATA%\PO_CONTAINS\part-00000-tid-2193262607082555932.csv ^
--relationships=PRODUCES_BATCH=%RELS_DATA%\PRODUCES_BATCH\part-00000-tid-2747029184498432977.csv ^
--relationships=SUPPL_BATCH=%RELS_DATA%\SUPPL_BATCH\part-00000-tid-3151750978191467391.csv ^
--relationships=MTLDOC_TRANSFERS_TO_BATCH=%RELS_DATA%\MTLDOC_TRANSFERS_TO_BATCH\part-00000-tid-7214036157690308425.csv ^
--relationships=TRANSFER_VIA_MATL_DOC=%RELS_DATA%\TRANSFER_VIA_MATL_DOC\part-00000-tid-6274627321655374374.csv ^
--relationships=TRANSFER_VIA_ORDER=%RELS_DATA%\TRANSFER_VIA_ORDER\part-00000-tid-2050133400947917221.csv ^
--trim-strings=true --legacy-style-quoting=true --ignore-empty-strings=true ^
--report-file=H:\test_data\load_test\bbn_import.txt ^
--skip-duplicate-nodes=true --skip-bad-relationships=true --bad-tolerance=999999999 --auto-skip-subsequent-headers=true ^
--verbose mydbname
```